Here is the complete, phased implementation workflow to upgrade the TraceText architecture to a high-performance, master-detail layout.
This plan optimizes text rendering to maintain the strict 60 FPS target while restructuring the UI so that complex regulatory or normative context is presented intuitively for non-technical end-users.

Phase 1: State & Concurrency Foundations
The first step is moving away from blocking operations on the GUI thread by establishing the communication channels and caching structures.

- **Define Communication Channels:** Create the ParserRequest and ParserResponse structs to facilitate message passing between the GUI and the worker thread.
- **Establish the In-Memory Cache:** Implement a two-tier caching strategy to prevent redundant I/O operations. You will need to define CachedDocument and CachedParagraph.
- **Update the Application State:** Add the Arc<RwLock<HashMap<PathBuf, CachedDocument>>> to your TraceTextGui struct alongside the Sender and Receiver channels.

Phase 2: The Background Processing Pipeline
To ensure the interface remains instantly responsive, heavy document extraction (unpdf / undoc) must be entirely offloaded.

- **Spawn the Worker Thread:** Inside TraceTextGui::new, initialize a std::thread::spawn loop that actively listens to rx_request.recv().
- **Implement Cache Checking:** When a request is received, the worker should first lock the RwLock and check if the document is already parsed. If a cache miss occurs, the worker executes the structural parsing and updates the HashMap.
- **Trigger UI Repaint:** Once the structured payload is transmitted back via tx_response.send(), the background thread must call ctx.request_repaint() to immediately wake the GUI thread and render the data.

Phase 3: Resizable Master-Detail UI Construction
Transitioning the layout involves replacing the standard vertical stack with a split view that automatically adjusts to viewport changes.

- **Implement the SidePanel:** Use the native egui::SidePanel::left().resizable(true) container to hold the search configuration and results table. This native approach automatically manages state persistence and layout phase complexity.
- **Calculate Dynamic Boundaries:** Retrieve the current window width using ui.available_rect_before_wrap().width() to restrict the left panel mathematically (e.g., between 35% and 65% of the total width).
- **Set up the CentralPanel:** Ensure the remaining screen real estate is consumed by egui::CentralPanel, which will host the document visualizer.

Phase 4: Table Interactivity & Context Visualization
The final phase wires the search results to the right-hand context view, utilizing Approach A (Structural Text Viewer) to avoid the immense memory overhead required by PDF page rasterization formulas like $Memory(Bytes)=Width(in)\times Height(in)\times DPI^{2}\times4~Bytes(RGBA)$.

- **Capture Row Selection:** Inside your egui_extras::TableBuilder body loop, use row.set_selected(is_selected) to handle the visual state. To detect user interaction, capture the union response of all cells by calling row.response() at the end of the row declaration.
- **Dispatch Data Requests:** When row_response.clicked() evaluates to true, update the selected_row_index and dispatch the ParserRequest to your background thread.
- **Render the Layout Jobs:** In the Central Panel, iterate over the cached CachedParagraph structures. Use egui::text::LayoutJob to apply specific styling (like a gold background with black text) to the matched substring indices while rendering the rest of the paragraph normally.